

Project Proposal: Proximal GaLore для памяти-эффективного обучения нейросетей

Дата: 26 апреля 2026

1. Название проекта

Proximal GaLore: динамическая адаптация ранга градиента через SVT в AdamW

2. Abstract

В проекте исследуется модификация GaLore-оптимизации: вместо фиксированного ранга проекции градиента используется проксимальный оператор ядерной нормы SVT, который автоматически выбирает эффективный ранг на каждом шаге пересчёта SVD. Цель — получить сопоставимое качество с AdamW и GaLore при меньшем расходе памяти и более устойчивом поведении без ручного подбора ранга. Прототип уже реализован в `galore_framework` и протестирован на MNIST на двух архитектурах с базовыми графиками динамики лосса, точности, ранга и памяти.

3. Описание проекта

Есть оптимизатор GaLore, который уменьшает память за счёт хранения состояний Adam в низкоранговом пространстве. Ограничение: ранг r фиксированный и чувствителен к подбору. Мы исследуем вариант, где:

- проекция градиента строится через SVD;
- сингулярные значения обрабатываются мягким порогом $\sigma_i \mapsto \max(\sigma_i - \lambda, 0)$;
- эффективный ранг выбирается автоматически и может меняться по ходу обучения.

4. Outcomes

1. **Код:**
 - реализация `ProximalGaLoreProjector` и `ProximalGaLoreAdamW`;
 - экспериментальный скрипт сравнения на MLP и CNN.
2. **Численные эксперименты:**
 - сравнение AdamW vs GaLore vs Proximal GaLore по качеству/памяти/скорости.
3. **Отчёт:**
 - литературный обзор;
 - анализ результатов и ограничений;
 - выводы, включая возможный отрицательный результат.
4. **Артефакты:**
 - графики из `galore/results/*`;

- итоговый PDF proposal/report.

5. Литературный обзор и научный ландшафт

В похожих постановках уже получены сильные результаты: Adam и AdamW задали базовый стандарт адаптивной оптимизации [2, 3], Adafactor показал возможность существенной экономии памяти за счёт факторизации второго момента [4], а Shampoo — эффективность матричного предобуславливания [5]. Для нашей темы ключевой опорой является GaLore [1], где память экономится через low-rank проекцию градиентов и хранение состояний оптимизатора в низкоранговом пространстве. Теоретический фундамент для нашего расширения даёт линия работ по ядерной норме и SVT как проксимальному оператору low-rank регуляризации [6, 7].

Более простыми по отношению к нашей идее являются GaLore с фиксированным рангом и Adafactor: они проще в настройке и реализации, но дают меньше гибкости при изменении структуры градиентов по ходу обучения. В Proximal GaLore мы делаем шаг в сторону адаптивности: вместо ручного фиксирования ранга используем мягкое отсечение сингулярных значений, чтобы эффективный ранг выбирался автоматически.

Актуальность задачи определяется ростом размеров моделей и ограничениями GPU-памяти: стоимость хранения состояний оптимизатора часто становится узким местом. Поэтому практически важен подход, который не ломает привычный пайплайн обучения и при этом даёт заметную экономию памяти. Для воспроизводимости проекта доступны открытые реализации: официальный репозиторий GaLore (<https://github.com/jiaweizzhao/GaLore>), стандартный `torch.optim.AdamW` в PyTorch и наш текущий прототип в папке `galore/` (файлы `experiment.py`, `galore_framework/projector.py`, `galore_framework/optimizers.py`).

6. Детальный план работ

Этап А — уже сделано

- Реализованы 3 оптимизатора: AdamW, GaLore AdamW, Proximal GaLore AdamW.
- Добавлен эксперимент на MNIST для двух архитектур: MLP и CNN.
- Построены графики потерь/точности/динамики аргумента/ранга/памяти.

Этап В — следующие 1–2 недели

- Провести sweep по `threshold`, `update_proj_gap`, `min_rank`.
- Добавить повторяемость по нескольким `seed` и таблицы со средним и стандартным отклонением.
- Выделить режимы, где proximal-версия лучше фиксированного ранга.

Этап С — далее

- Увеличить сложность задач: FashionMNIST и CIFAR-10 на той же методике.
- Сравнить стабильность при разных шагах обучения.
- Сформулировать практические рекомендации по выбору гиперпараметров.

Этап D — финал

- Подготовить финальный отчёт и аккуратные иллюстрации высокого качества.
- Описать ограничения и потенциальные направления продолжения.

7. Метрики качества

Формальные, измеряемые метрики:

1. **Качество модели:** test accuracy (%), финальный train loss.
2. **Сходимость:** площадь под кривой loss vs steps/time, число шагов до заданного accuracy.
3. **Память оптимизатора:** MB состояния оптимизатора (compute_memory_footprint).
4. **Стабильность:** std метрик по нескольким seed.
5. **Структурная метрика метода:** средний/медианный эффективный ранг в Proximal GaLore.

Критерии успеха:

- test accuracy не хуже GaLore более чем на 1–2 процентных пункта;
- память состояния меньше AdamW и не хуже GaLore в ряде режимов;
- наблюдаемая адаптация ранга, когда ранг не «залипает» в константу во всех экспериментах.

8. Отчёт о фазе прототипирования

Что уже запущено

- Основной скрипт: `python3 experiment.py`.
- Датасет: MNIST.
- Архитектуры: MLP и CNN.
- Методы: AdamW, GaLore, Proximal GaLore.

Что получено

- Сгенерированы графики для анализа:
 - `results/mlp_benchmark.png`
 - `results/cnn/cnn_benchmark.png`
 - `results/loss_curves.png`, `results/rank_evolution.png`,
`results/memory_footprint.png`, `results/singular_values.png`.
- Проверена работоспособность динамического ранга в проекторе `ProximalGaLoreProjector`.

С какими проблемами столкнулись

- Чувствительность к `threshold`: слишком высокий порог может быстро зажать ранг.
- SVD пересчёт влияет на время шага, особенно при частом `update_proj_gap`.
- Для честного сравнения нужно больше повторов по нескольким `seed`, а это увеличивает время экспериментов.

Первые выводы

- Идея с проксимальным порогом технически реализуема и уже работает на полном цикле обучения.
 - Следующий ключевой вопрос: где проходит граница «экономия памяти vs потеря качества» относительно фиксированного ранга.
-

9. Литература

- [1] Zhao J., Li Y., Zhang M. et al. GaLore: Memory-Efficient LLM Training by Gradient Low-Rank Projection. arXiv:2403.03507, 2024. <https://arxiv.org/abs/2403.03507>
- [2] Kingma D. P., Ba J. Adam: A Method for Stochastic Optimization. ICLR, 2015. <https://arxiv.org/abs/1412.6980>
- [3] Loshchilov I., Hutter F. Decoupled Weight Decay Regularization. ICLR, 2019. <https://arxiv.org/abs/1711.05101>
- [4] Shazeer N., Stern M. Adafactor: Adaptive Learning Rates with Sublinear Memory Cost. ICML, 2018. <https://proceedings.mlr.press/v80/shazeer18a.html>
- [5] Gupta V., Koren T., Singer Y. Shampoo: Preconditioned Stochastic Tensor Optimization. ICML, 2018. <https://proceedings.mlr.press/v80/gupta18a.html>
- [6] Cai J.-F., Candes E. J., Shen Z. A Singular Value Thresholding Algorithm for Matrix Completion. SIAM Journal on Optimization, 20(4):1956-1982, 2010. <https://doi.org/10.1137/080738970>
- [7] Recht B., Fazel M., Parrilo P. A. Guaranteed Minimum-Rank Solutions of Linear Matrix Equations via Nuclear Norm Minimization. SIAM Review, 52(3):471-501, 2010. <https://doi.org/10.1137/070697835>
- [8] Hu E. J., Shen Y., Wallis P. et al. LoRA: Low-Rank Adaptation of Large Language Models. ICLR, 2022. <https://arxiv.org/abs/2106.09685>